Git & GitHub Commands (Production Cheat Sheet)

⚠️ This is a *production-focused* reference, but Git and GitHub have too many commands and flags to list literally *all* of them in one page. Use `git help -a` to list every Git subcommand installed on your machine, and `git help <command>` for the complete manual.

How to read this page

- **Command** blocks show the exact syntax.
- *When to use* explains the typical production scenario.
- Tips include safe defaults and common gotchas.

---

1) Setup and identity

`git config`

When to use: set identity, editor, line endings, aliases, and safe defaults.

```
# Identity
git config --global user.name "Your Name"
git config --global user.email "you@example.com"

# Useful defaults
git config --global init.defaultBranch main
git config --global pull.rebase false      # or true if your
team uses rebase pulls
git config --global fetch.prune true

# Tooling
git config --global core.editor "code --wait"  # VS Code
```

```
# See config
git config --list --show-origin
```

Tip: Prefer repo-local config (omit `--global`) for team-specific behavior.

2) Create and get code

`git init`

When to use: create a new repository.

```
git init
```

`git clone`

When to use: copy an existing repository.

```
# HTTPS
git clone https://github.com/OWNER/REPO.git

# SSH
git clone git@github.com:OWNER/REPO.git

# Clone a specific branch
git clone -b main --single-branch https://github.com/OWNER/RE
PO.git
```

3) Day-to-day work (local changes)

`git status`

When to use: see what changed and what will be committed.

```
git status
```

`git add`

When to use: stage changes for a commit.

```
# Stage specific files
git add file1 file2

# Stage everything in the working tree
git add .

# Stage parts of a file interactively
git add -p
```

`git diff`

When to use: review changes before staging or committing.

```
# Unstaged changes
git diff

# Staged changes
git diff --staged

# Compare with another branch
git diff main..feature/my-work
```

`git restore` (newer) and `git checkout --` (older)

When to use: discard local changes safely.

```
# Discard unstaged changes in a file
git restore file

# Unstage a file (keep local changes)
git restore --staged file
```

```
# Restore from a specific commit
git restore --source <commit> -- path/to/file
```

`git rm` and `git mv`

When to use: delete or rename files while tracking history.

```
# Remove and stage deletion
git rm path/to/file

# Keep file in working tree but remove from index
git rm --cached path/to/file

# Rename/move
git mv oldname newname
```

---

4) Commits

`git commit`

When to use: record a snapshot of staged changes.

```
# Commit with message
git commit -m "feat: add user audit logging"

# Open editor for a detailed message
git commit

# Amend last commit (edit message and/or include staged chang
es)
git commit --amend
```

Tip: Avoid `--amend` on commits already pushed to shared branches.

`git log`

When to use: inspect history.

```
# One-line graph
git log --oneline --decorate --graph --all

# Show file history with diffs
git log -p -- path/to/file
```

`git show`

When to use: inspect a single commit, tag, or object.

```
git show <commit>
```

---

5) Branching

`git branch`

When to use: list, create, rename, delete branches.

```
# List local branches
git branch

# Create branch
git branch feature/my-work

# Rename current branch
git branch -m new-name

# Delete local branch
git branch -d feature/my-work   # safe delete
git branch -D feature/my-work   # force delete
```

`git switch` (newer) and `git checkout`

When to use: move between branches.

```
# Switch to an existing branch
git switch main

# Create and switch
git switch -c feature/my-work

# Older equivalent
git checkout -b feature/my-work
```

`git merge`

When to use: integrate another branch.

```
# Merge feature into current branch
git merge feature/my-work

# Create a merge commit even if fast-forward is possible
git merge --no-ff feature/my-work

# Abort a conflicted merge
git merge --abort
```

`git rebase`

When to use: replay commits to keep a linear history.

```
# Rebase current branch on top of main
git fetch origin
git rebase origin/main

# Interactive rebase (rewrite last N commits)
git rebase -i HEAD~5

# Abort rebase
git rebase --abort
```

Warning: Do not rebase commits that others may have pulled.

`git cherry-pick`

When to use: bring a specific commit into another branch (hotfixes).

```
git cherry-pick <commit>

# Continue after resolving conflicts
git cherry-pick --continue

# Abort
git cherry-pick --abort
```

---

6) Remote repositories

`git remote`

When to use: manage remote URLs.

```
# List remotes
git remote -v

# Add remote
git remote add origin https://github.com/OWNER/REPO.git

# Change URL
git remote set-url origin git@github.com:OWNER/REPO.git
```

`git fetch`

When to use: download remote history without changing your branch.

```
git fetch origin

git fetch --all --prune
```

`git pull`

When to use: fetch + integrate into current branch.

```
# Merge-based pull
git pull origin main

# Rebase-based pull
git pull --rebase origin main
```

`git push`

When to use: publish commits.

```
# Push current branch
git push

# Push and set upstream
git push -u origin feature/my-work

# Delete remote branch
git push origin --delete feature/my-work

# Push tags
git push --tags
```

---

7) Tags and releases

`git tag`

When to use: mark release points.

```
# Lightweight tag
git tag v1.2.3

# Annotated tag (recommended)
```

```
git tag -a v1.2.3 -m "Release v1.2.3"

# List tags
git tag --list

# Push a tag
git push origin v1.2.3
```

8) Stash and work-in-progress

`git stash`

When to use: temporarily shelve changes.

```
# Stash tracked changes
git stash

# Stash including untracked files
git stash -u

# List stashes
git stash list

# Apply latest stash (keep it)
git stash apply

# Pop latest stash (remove it)
git stash pop

# Drop a stash
git stash drop stash@{0}
```

9) Undo and recovery (production-safe patterns)

`git revert`

When to use: safely undo a pushed commit by creating a new commit.

```
# Revert a commit
git revert <commit>

# Revert a merge commit (choose mainline parent)
git revert -m 1 <merge-commit>
```

`git reset`

When to use: rewrite local history (be careful).

```
# Keep changes in working tree, move HEAD back
git reset --mixed HEAD~1

# Keep changes staged
git reset --soft HEAD~1

# Discard changes (dangerous)
git reset --hard HEAD~1
```

`git reflog`

When to use: recover “lost” commits after reset/rebase.

```
git reflog
# Then:
git checkout <reflog-commit>
```

---

10) Conflict resolution

`git mergetool`

When to use: resolve conflicts using a configured tool.

```
git mergetool
```

Common conflict workflow

```
git status
# edit files to resolve conflicts
git add <resolved-files>

git merge --continue   # if merging
# or
git rebase --continue  # if rebasing
```

---

11) Inspection, blame, and debugging

`git blame`

When to use: identify who last changed each line.

```
git blame path/to/file
```

`git bisect`

When to use: find the commit that introduced a bug.

```
git bisect start
git bisect bad
git bisect good <commit-or-tag>
# test each step and mark good/bad until found

git bisect reset
```

`git grep`

When to use: search tracked code fast.

```
git grep "someSymbol"
```

12) Cleaning

`git clean`

When to use: remove untracked files and directories.

```
# Preview first!
git clean -n

# Remove untracked files
git clean -f

# Remove untracked files + directories
git clean -fd
```

13) Submodules (when teams use them)

`git submodule`

When to use: manage nested repositories.

```
# Clone repo with submodules
git clone --recurse-submodules https://github.com/OWNER/REPO.
git

# Init and update in an existing clone
git submodule update --init --recursive

# Update to latest remote-tracking commits (if configured)
git submodule update --remote --merge
```

GitHub (platform) commands

🐙 These are the most common GitHub CLI commands used in production workflows. They require the GitHub CLI: `gh`.

14) Authentication and context

`gh auth`

```
# Log in (browser)
gh auth login

# Check auth status
gh auth status
```

`gh repo`

```
# Clone (GitHub-aware)
gh repo clone OWNER/REPO

# Create a new repo from current directory
gh repo create
```

---

15) Pull requests (PRs)

`gh pr`

```
# Create a PR (interactive)
gh pr create

# List PRs
gh pr list
```

```
# View PR details
gh pr view <number>

# Check out a PR locally
gh pr checkout <number>

# Merge a PR
gh pr merge <number>

# Status across repos
gh pr status
```

---

16) Issues

`gh issue`

```
# Create issue
gh issue create

# List issues
gh issue list

# View issue
gh issue view <number>
```

---

17) Workflows and CI/CD

`gh workflow` and `gh run`

```
# List workflows
gh workflow list

# Run a workflow
gh workflow run <workflow-name-or-id>
```

```
# List workflow runs
gh run list

# Watch a run
gh run watch <run-id>

# View logs
gh run view <run-id> --log
```

18) Releases

`gh release`

```
# Create a release from a tag
gh release create v1.2.3 --title "v1.2.3" --notes "Release no
tes here"

# List releases
gh release list

# View a release
gh release view v1.2.3
```

19) GitHub Actions (YAML) quick template

```
name: CI

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]
```

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up Node
        uses: actions/setup-node@v4
        with:
          node-version: "20"
      - name: Install
        run: npm ci
      - name: Test
        run: npm test
```

---

20) “Show me everything installed” (no missing commands)

List all Git subcommands available on your machine

```
git help -a
```

Full docs for a command

```
git help <command>
# Example:
git help rebase
```